

Intermediate (build real memory) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and the classic bug](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the classic bug

Basic Version

There's no built-in "memory" to turn on. You build memory yourself, by keeping a growing list of every message so far, and sending the *whole* list again every time you call the model.

Start with a list that has just your system message in it. Each time around your loop: add the user's new message to the list, send the whole list, get a reply, add the reply to the list too.

Watch out for this exact bug: forgetting to append the assistant's reply back into the history list. If you skip that step, the model never sees its own previous answers — it'll look like it "forgot" everything within a couple of turns, and it's the single most common mistake here.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The whole exercise is one idea: a Python list, mutated in place, that grows by exactly two entries per turn — one `{"role": "user", ...}` and one `{"role": "assistant", ...}`. Every single call sends this entire list, not just the newest message; the model has no server-side memory of its own (this is the exact "No memory" idea from Doc03).

The response object doesn't give you back a ready-made message dict — you build one yourself from the reply text: `{"role": "assistant", "content": response.choices[0].message.content}`. Append this to `history`, not just print it, or the next turn's call won't include it.

Also decide how the loop ends — a sentinel input like typing `"quit"` is the simplest approach: check for it right after reading input, before appending anything, so the exit command itself never becomes part of the conversation history sent to the model.

Difference between Basic and Intermediate: Basic and Intermediate both build memory correctly for one conversation, one script, one `history` list — Intermediate names the exact pieces (the message-dict shape, the exit condition) more precisely, in a reusable function.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
history = [system message]

loop forever:
    read user_input
    if user_input is "quit": stop the loop

    add {user, user_input} to history
    send history to the model, get reply
    add {assistant, reply} to history
    print reply
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# conversation_memory_practice.py
history = [
    {"role": "system", "content": "You are a helpful assistant."}
]

while True:
    user_input = input("You: ")
    if user_input == "quit":
        break

    history.append({"role": "user", "content": user_input})

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
    )
    reply = response.choices[0].message.content

    history.append({"role": "assistant", "content": reply})
    print("Assistant:", reply)
```

Expected behavior: tell it your favorite color in turn 1, ask "what's my favorite color?" in turn 2 — if `history` is being built and sent correctly, turn 2's answer references it correctly.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
function get_reply(history) -> str:
    call the API with messages=history
    return the reply text

function run_chat():
    history = [system message]
    loop:
        read user_input; if "quit": stop
        history.append(user message)
        reply = get_reply(history)
        history.append(assistant message)
        print reply
```

```
# conversation_memory_practice.py
def get_reply(history: list) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
    )
    return response.choices[0].message.content

def run_chat() -> None:
    history = [
        {"role": "system", "content": "You are a helpful assistant."}
    ]

    while True:
        user_input = input("You: ")
        if user_input == "quit":
            break

        history.append({"role": "user", "content": user_input})
        reply = get_reply(history)
        history.append({"role": "assistant", "content": reply})

        print("Assistant:", reply)
```

Trace through 2-3 turns by hand and confirm `history` has the length you expect after each one, then write the `client` /import lines yourself, then compare against the [Solution](#).

Difference between Basic and Intermediate: Basic and Intermediate both build one correct, growing `history` list for one conversation. Intermediate wraps the API call in a named, typed function that the Build Task's own `send_message()` copies the shape of.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)